Network Services

Lecture 1 - Chapter 25

Data Communications and Networking, 5th edition, 2013.

1) Service Concept

All communication networks that started before the Internet were designed to provide services to network users.

For example, the **telephone** network was originally designed to provide **voice service**.

The **Internet** was originally designed for the same purpose: to provide service to users around the world.

Since the **application layer** is the only layer that provides services to the Internet user through its **protocols**.

2) Standard and Non-Standard Protocols

The application-layer protocols can be both standard and nonstandard.

Standard Application-Layer Protocols

Standard protocols: are protocols **standardized** and **documented** by the **Internet authority** which is **Internet Assigned Numbers Authority** (**IANA**).

IANA is owned by **Internet Corporation for Assigned Names and Numbers** (**ICANN**).

Each standard protocol is a **pair** of computer programs that interact with the user and the transport layer to provide a specific service to the user.

Nonstandard Application-Layer Protocols

Two programs are written by a **programmer** that provide service to the user by interacting with the transport layer.

It is the creation of a nonstandard (**proprietary**) protocol, which does not even need the **approval** of the Internet authorities if **privately** used, that has made the Internet so popular worldwide.

What is needed is to **write** programs, in one of the **computer languages**, that use the available **services** provided by the transport-layer protocols TCP or UDP.

3) Application Layer Services Paradigm

It should be clear that to use the Internet we need **two** application programs to interact with each other:

One running on a computer somewhere in the world, the other running on another computer somewhere else in the world.

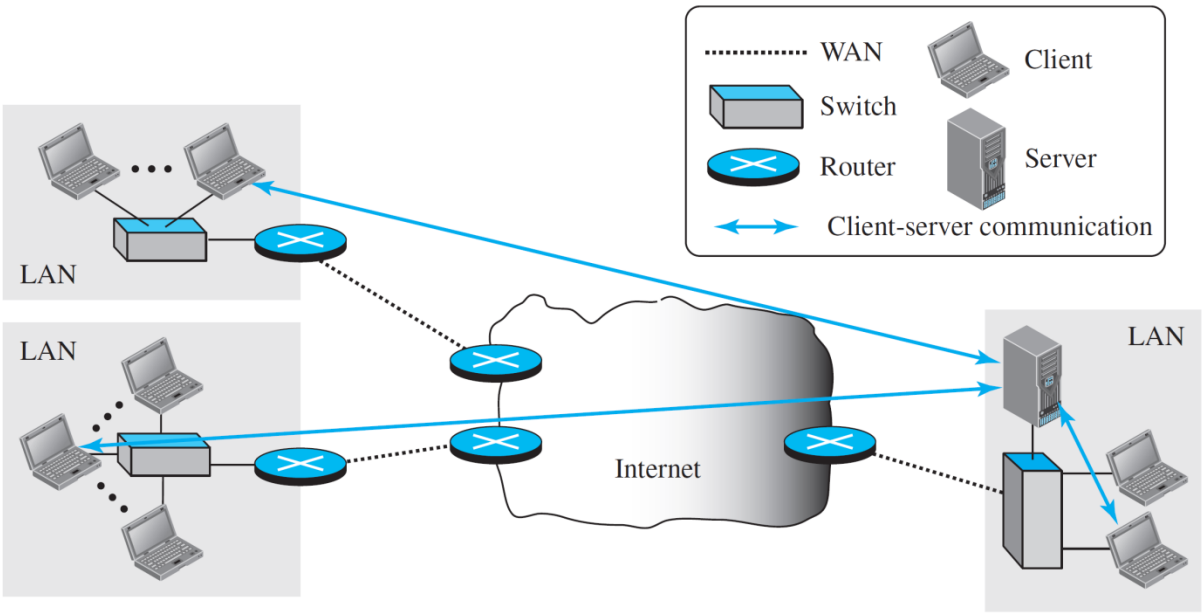
Two paradigms of the application programs have been developed during the lifetime of the Internet: the ***client-server*** *paradigm* and the ***peer-to-peer*** *paradigm.*

Client-server paradigm

The **traditional** paradigm is called the **client-server paradigm**. It was the most popular paradigm until a few years ago.

In this paradigm, the **service provider** is an application program, called the **server** process; it runs **continuously**, **waiting** for another application program, called the **client** process, to make a connection through the Internet and **ask** for service.

Figure 25.2 *Example of a client-server paradigm*



Problems of this paradigm are:

a) The **concentration** of the communication load is on the **shoulder of the server**, which means the server should be a **powerful** computer.

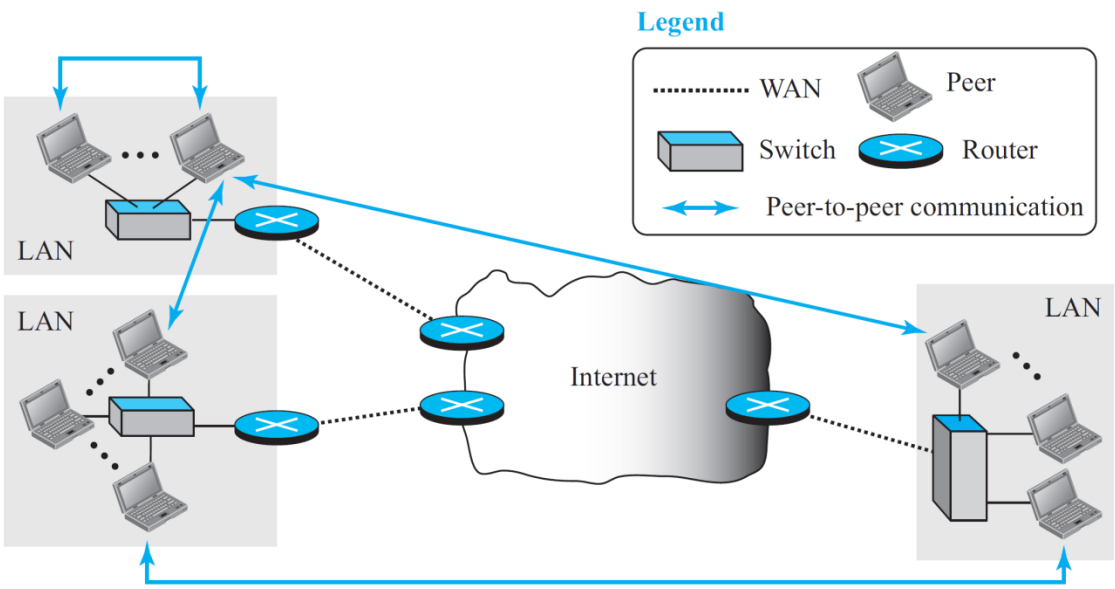
b) There should be a **service provider** willing to accept the **cost** and **create** a powerful server for a specific service, which means the service must always return some type of **income** for the server in order to encourage such an arrangement.

peer-to-peer paradigm

A new paradigm, called the **peer-to-peer paradigm** (often abbreviated ***P2P*** *paradigm*) has emerged to respond to the needs of some new applications.

In this paradigm, there is **no need** for a server process to be running all the time and waiting for the client processes to connect. The responsibility is **shared** between peers.

Figure 25.3 *Example of a peer-to-peer paradigm*



The peer-to-peer paradigm can be **used** when some **computers** connected to the Internet have something to **share** with each other.

Although the peer-to-peer paradigm has been proved to be easily **scalable** and **cost-effective** in **eliminating** the **need** for **expensive servers** to be running and maintained all the time.

There are also some challenges.

The main challenge are:

1) **Security**; it is more difficult to create secure communication between **distributed**
services than between those controlled by some **dedicated** servers.

2) The other challenge is applicability; it appears that not all applications can use this new paradigm. For example, not many Internet users are ready to become **involved**, if one day the Web can be implemented as a peer-to-peer service.

There are some new applications, such as BitTorrent, Skype, IPTV, and Internet telephony, that use this paradigm.

Mixed Paradigm

An application may choose to use a **mixture** of the two paradigms by combining the advantages of both.

For example, a **light-load** client-server communication can be used to find the address of the peer that can offer a service. When the **address** of the peer is **found**, the actual service can be received from the peer by using the peer-to-peer paradigm.

4) Client-Server Programming

In a client-server paradigm, communication at the application layer is between two running application programs called processes: a client and a server.

A client is a running program that **initializes** the communication by sending a request.

A server is another application program that **waits** for a **request** from a client.

The server **handles** the **request** received from a client, prepares a **result**, and **sends** the result back to the client.

We need to be careful that the server program is **started before** we start running the client program.

Application Programming Interface

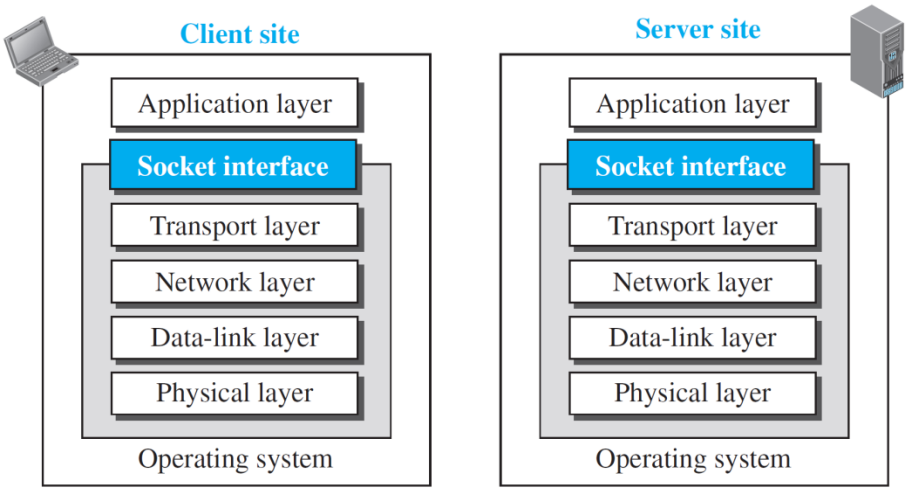
If we need a process to be able to communicate with another process, we need a new **set of instructions** to tell the lowest four layers of the TCP/IP suite to **open** the connection, **send** and **receive** data from the other end, and **close** the connection.

A set of instructions of this kind is normally referred to as an **application programming interface (API)**. An interface in programming is a set of instructions between two entities.

Several APIs have been designed for communication. Three among them are common: **socket interface, Transport Layer Interface (TLI),** and **STREAM**. In this chapter, we briefly discuss only *socket interface*, the most common one.

Socket interface started in the early 1980s at UC Berkeley as part of a UNIX environment. The **socket interface** is a set of instructions that provide communication between the **application layer** and the **operating system**.

Figure 25.4 *Position of the socket interface*



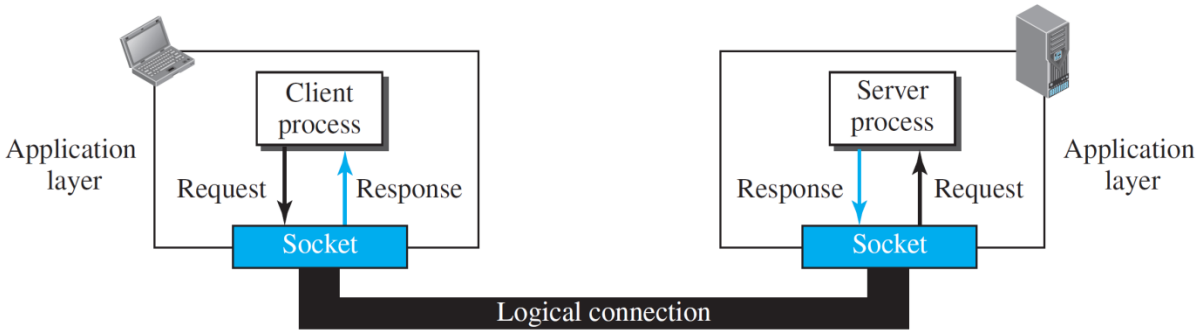
Sockets

The idea of **sockets** allows us to use the set of all instructions already designed in a programming language for other **sources** and **sinks**.

The client thinks that the socket is the entity that receives the request and gives the response;

The server thinks that the socket is the one that has a request and needs the response.

Figure 25.6 *Use of sockets in process-to-process communication*



Socket Addresses

A **socket address** should first define the computer on which a client or a server is running.

A computer in the Internet is uniquely defined by its IP address, a 32-bit integer in the current Internet version.

However, **several** client or server processes may be **running** at the same time on a computer, which means that we need another identifier to define the specific client or server involved in the communication.

An application program can be defined by a **port** number, a 16-bit integer.

This means that a socket address should be a **combination** of an IP address and a port.

Figure 25.7 *A socket address*

Finding Socket Addresses

How can a client or a server find a pair of socket addresses for communication?

Server Site

Local Socket Address The local (server) socket address is provided by the operating system.

Remote Socket Address The remote socket address for a server is the socket address of the client that makes the connection. The server can find this socket address when a client tries to connect to the server.

Client Site

The client also needs a local (client) and a remote (server) socket address for communication.

Local Socket Address The local (client) socket address is also provided by the operating system.

Remote Socket Address Finding the remote (server) socket address for a client, however, needs **more work**. When a client process starts, it should know the socket address of the server it wants to connect to. We will have two situations in this case.

Sometimes, the **user** who **starts** the client process **knows** both the server port number and IP address of the computer on which the server is running.

Although each standard application has a **well-known port** number, most of the time, we do not know the **IP address**.

Iterative Communication Using UDP

Communication between a client program and a server program can occur **iteratively** or **concurrently**.

An **iterative** server can process one client request at a time; it receives a request, processes it, and sends the response to the requestor before handling another request.

A **concurrent** server can process several client requests at the same time.

Figure 25.8 *Sockets for UDP communication*

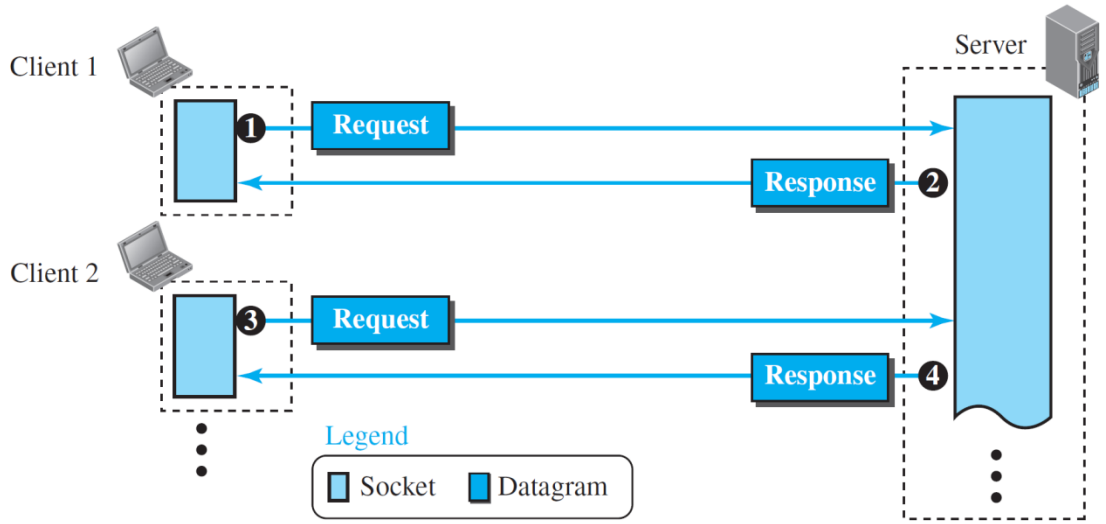
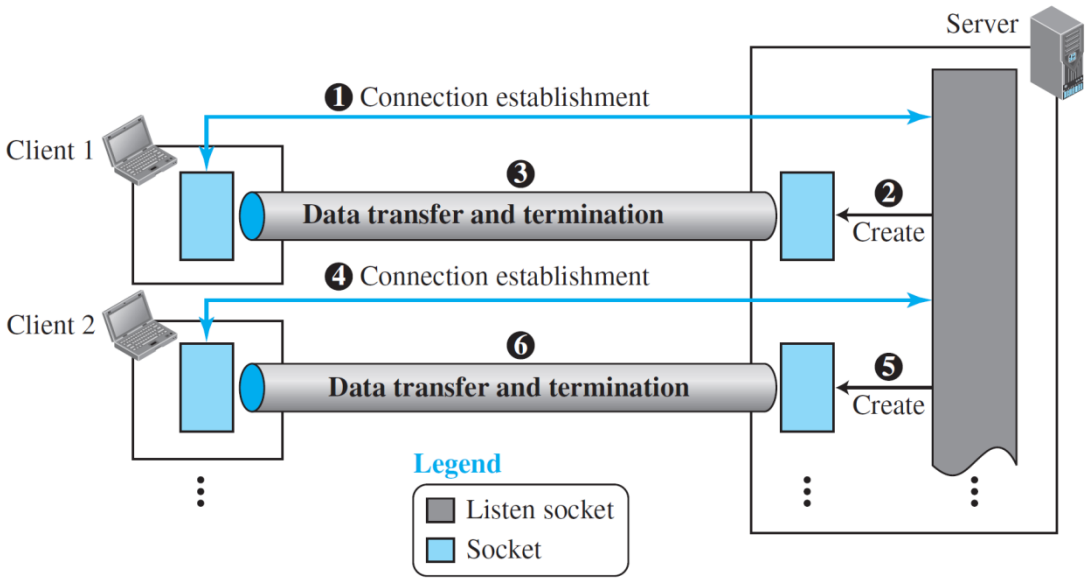



Figure 25.10 *Sockets used in TCP communication*